

Heaps & Top-K

Heap ఒక sorted జాబితా కాదు. అది "అతి చిన్నది పైన" అనే ఒక్క ఆస్తి మాత్రమే ఇస్తుంది – మరియు ఆ తక్కువ హామీయే దాని వేగం.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది – 10 లక్షల నుంచి అతి పెద్ద 100: sort చేస్తే 289 ms, heap తో **4.1 ms** (70 రెట్లు). రెండేది – "పెద్దవి కావాలి కదా" అని max-heap వాడితే **65.8% తప్పు**. మూడేది – quickselect ఇప్పటికే sorted input మీద **1,189 రెట్లు** ఎక్కువ పని.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే – timings తో సహా.

HEAP యొక్క మొత్తం ఆలోచన ఒక్క వాక్యంలో

Sorting మీకు అవసరం లేని సమాచారాన్ని లెక్కిస్తుంది.

10 లక్షల మూలకాల్లో అతి పెద్ద 10 కావాలంటే – మిగతా 9,99,990 మూలకాల మధ్య క్రమం మీకు **ఎందుకూ పనికిరాదు**. కానీ `sort()` దాన్ని కూడా లెక్కిస్తుంది.

Heap ఒక్క ప్రశ్నే జవాబిస్తుంది: "ఇప్పటిదాకా చూసిన వాటిలో అతి చిన్నది ఏది?" – మరియు ఆ ఒక్క ప్రశ్నకి $O(1)$, నవీకరణకి $O(\log n)$.

§4 lo కొలుస్తాం: **70 రెట్లు**. మరియు §5 lo – $k = 1$ అయితే ఒక సాధారణ loop heap ని ఓడిస్తుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Heap ఏ ప్రశ్నకి జవాబిస్తుంది
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: అన్నీ sort చేయడం

3. Step – అతి పెద్ద k కి మూడు విధానాలు
4. మొదటి విరుపు – 289 ms vs 4.1 ms
5. Step – కానీ $k = 1$ అయితే loop గెలుస్తుంది

Part 3 – రెండో విరుపు: ఏ heap

6. Step – k -పరిమాణ heap యొక్క తర్కం
7. రెండో విరుపు – max-heap వాడితే 65.8% తప్పు

Part 4 – మూడో విరుపు: pivot

8. Step – quickselect · heap అవసరం లేని దారి
9. Step – pivot ఎంపిక
10. మూడో విరుపు – sorted input మీద 1,189 రెట్లు

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code
12. 87,647 pushes · mutation testing · పనిని ఒక నియమంగా

13. బతికిన మార్పు – నకళ్ళు మరియు మూడు-ముక్కల విభజన
14. Follow-ups – heapify, median stream, k-way merge
15. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Heap ఏ ప్రశ్నకి జవాబిస్తుంది

Heap ఒక array, మరియు దానికి ఒకే ఒక నియమం:

ప్రతి node తన పిల్లల కంటే చిన్నది (min-heap) – లేదా పెద్దది (max-heap).

అంతే. **తోబుట్టువుల మధ్య ఏ క్రమమూ లేదు.** [1, 5, 3, 9, 7] ఒక చెల్లుబాటు min-heap, మరియు అది sorted కాదు.

ఆ "తక్కువ హామ్" యే వేగాన్ని ఇస్తుంది

పూర్తిగా sort చేయాలంటే n మూలకాల మధ్య అన్ని సంబంధాలూ తేల్చాలి – $O(n \log n)$, మరియు దాన్ని తగ్గించలేము.

Heap కి కావాల్సింది ఒక్క దారి మాత్రమే సరిగ్గా ఉండటం: మూలం నుంచి కిందికి. కాబట్టి ఒక మూలకాన్ని చేర్చడం/తీసేయడం ఒక దారి పొడవు = $O(\log n)$.

మీరు తక్కువ అడిగారు, కాబట్టి తక్కువ చెల్లించారు. ఈ series lo ఇదే ఆలోచన పదే పదే వస్తుంది.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
k ఎంత – n తో పోలిస్తే?	k చిన్నదైతే heap; $k \approx n$ అయితే sort చేయడమే మెరుగు (§4)
అతి పెద్దవా, అతి చిన్నవా?	ఇదే §7 – మరియు జవాబు మీరు అనుకున్నదానికి వ్యతిరేకం
Data ఒకేసారి వస్తుందా, ప్రవాహంగానా?	ప్రవాహం అయితే heap తప్పనిసరి – quickselect కి మొత్తం array కావాలి
క్రమం కావాలా, లేక సమితి చాలా?	సమితి చాలంటే quickselect $O(n)$ (§8)

ప్రశ్న	ఎందుకు అడుగుతున్నాం
నకళ్ళు ఎక్కువా?	అవును అంటే §13 – ఒక్క-pivot విభజన చతుర్భుజం అవుతుంది
k-వ మూలకం ఒకటే కావాలా?	అవును అంటే heap అనవసరం – quickselect

ఒక ప్రశ్న అడిగితే మేరు దీన్ని నిజంగా రాశారని తెలుస్తుంది

"Data ఒకేసారి వస్తుందా, లేక ప్రవాహంగానా?"

ఎందుకంటే ఇది **రెండు పూర్తిగా వేర్వేరు** పరిష్కారాలని వేరు చేస్తుంది:

ఒకేసారి → quickselect, సగటున $O(n)$, heap కంటే వేగం.

ప్రవాహం → quickselect **అసాధ్యం** (దానికి మొత్తం array కావాలి, మరియు అది array ని మారుస్తుంది). k-పరిమాణ heap ఒక్కటే దారి, మరియు అది **$O(k)$ memory** లో పనిచేస్తుంది.

"రోజుకి కోట్ల events lo అతి పెద్ద 100" – ఇది ప్రవాహం. మొత్తం data ని ఉంచడం సాధ్యం కాదు.

Part 2 – మొదటి విరుపు: అన్నీ sort చేయడం

3. Step – అతి పెద్ద k కి మూడు విధానాలు

```
const bySort = (a,k) => Array.from(a).sort((x,y)=>y-x).slice(0,k); // అన్నీ sort

const byHeap = (a,k) => { const h = new MinHeap(); // k-పరిమాణ heap
  for (const x of a) { if (h.size < k) h.push(x);
    else if (x > h.peek()) { h.pop(); h.push(x); } }
  return h.a.slice().sort((x,y)=>y-x); };

const byScan = (a,k) => { /* k సార్లు మొత్తం array ని స్కాన్ */ }; // O(nk)
```

4. మొదటి విరుపు – 289 ms vs 4.1 ms

n = 10 లక్షల మూలకాల నుంచి అతి పెద్ద k · 3 runs to ఉత్తమమైనది

k	అన్నీ sort చేసి	k-పరిమాణ heap	k సార్లు స్కాన్	heap ఎన్ని రెట్లు వేగం
1	287.6 ms	5.8 ms	2.4 ms	
49.75x				
10	285.5 ms	4.2 ms	11.2 ms	
68.57x				
100	288.8 ms	4.1 ms	111.7 ms	
70.45x				
10,000	296.8 ms	10.0 ms	–	
29.64x				
1,00,000	287.4 ms	55.9 ms	–	
5.14x				

K = 100 దగ్గర 70 రెట్లు · SORT చేసిన సమయం K తో మారనే లేదు

sort() నిలుపు వరుస మొత్తం **287-297 ms** – k ఏదైనా సరే. ఎందుకంటే sort క్రమం మొత్తం లెక్కిస్తుంది, ఆపై మనం **0.01%** తీసుకుని మిగతాది పడేస్తాం.

Heap k తో పెరుగుతుంది: 4.1 ms → 55.9 ms. కాబట్టి k పెరిగే కొద్దీ లాభం తగ్గుతుంది – 70x నుంచి 5x.

k ≈ n అయితే sort చేయడమే మెరుగు, ఎందుకంటే అప్పుడు మీరు నిజంగా మొత్తం క్రమాన్నే అడుగుతున్నారు.

5. Step – కానీ $k = 1$ అయితే loop గెలుస్తుంది

మొదటి వరుస మళ్ళీ చూడండి:

1	287.6 ms	5.8 ms	2.4 ms
---	----------	--------	--------

K = 1 దగ్గర సాధారణ స్కాన్ (**2.4 MS**) HEAP (**5.8 MS**) కంటే **2.4** రెట్లు వేగం

$k = 1$ అంటే "గరిష్టం కనుగొను" – దానికి ఒక `for` loop మరియు ఒక `if` చాలు. Heap ఒక object, పోలికలు, method calls.

కానీ $k = 100$ దగ్గర ఆ స్కాన్ **111.7 ms** – heap కంటే **27 రెట్లు** నెమ్మది, ఎందుకంటే అది $O(nk)$.

Deep 01 lo $n \leq 32$, **Deep 03** lo Set, **Deep 04** lo చిన్న memo – ఇప్పుడు $k = 1$. ఐదే doc lo నాలుగోసారి: సరళమైనది తరచుగా వేగం, మరియు అది ఎప్పుడో తెలుసుకోవాలి.

Part 3 – రెండో విరుపు: ఏ heap

6. Step – k-పరిమాణ heap యొక్క తర్కం

అతి పెద్ద k ని ఉంచాలి, మరియు heap lo k మూలకాలే ఉండాలి. కొత్త మూలకం వచ్చినప్పుడు:

heap నిండి ఉంది → ఒకదాన్ని తొలగించాలి → *ఏ* దాన్ని?

మనం ఉంచాలనుకుంటున్నది అతి పెద్దవి. కాబట్టి తొలగించాల్సింది heap lo అతి చిన్నది.

మరియు heap పైన ఉండేది మనం తొలగించేది. కాబట్టి **heap పైన అతి చిన్నది** ఉండాలి – అంటే **min-heap**.

7. రెండో విరుపు – max-heap వాడితే 65.8% తప్పు

అతి పెద్ద k ని కనుగొనడం • 1,00,000 యాదృచ్ఛిక cases

ఏ heap	తప్పు జవాబులు	%
min-heap, కనిష్టాన్ని తొలగించడం (సరైనది)	0	0.0%
max-heap ("పెద్దవి కావాలి కదా")	65,752	65.8%

ఉదా: a=[38,10,31,4,29,36,22,45,11] k=4 → max-heap [45,31,10,4], నిజం [45,38,36,31]

MAX-HEAP జవాబు [45, 31, 10, 4] – అందులో 4 ఉంది

అది array lo అతి చిన్న మూలకం. "అతి పెద్ద 4" అనే జాబితాలో.

ఎందుకంటే max-heap పైన **అతి పెద్దది** ఉంటుంది, మరియు code ఆ పైనున్నదాన్ని **తొలగిస్తుంది**. కాబట్టి ప్రతి మంచి మూలకం వచ్చినప్పుడు, ఇప్పటిదాకా ఉన్న **అత్యుత్తమమైనది** బయటికి వెళ్ళిపోతుంది.

ఇది ఒక తప్పు కాదు – ఇది సరిగ్గా వ్యతిరేకం.

గుర్తుపెట్టుకోవడానికి ఒక వాక్యం

"Heap పైన ఉండేది మేరు బయటికి పంపేది, మేరు ఉంచేది కాదు."

అతి పెద్ద k కావాలి → అతి చిన్నదాన్ని పంపాలి → **min-heap**.

అతి చిన్న k కావాలి → అతి పెద్దదాన్ని పంపాలి → **max-heap**.

ఇది "వ్యతిరేకం" అనిపిస్తుంది, మరియు అందుకే 65.8%.

Part 4 – మూడో విరుపు: pivot

8. Step – quickselect · heap అవసరం లేని దారి

k-వ మూలకం ఒక్కటే కావాలంటే (జాబితా కాదు), heap కూడా అవసరం లేదు:

```
function quickselect(a, k, rand) {
  let lo = 0, hi = a.length - 1;
  while (lo < hi) {
    const p = /* ఒక pivot ఎంచుకో */;
    [a[p], a[hi]] = [a[hi], a[p]];
    let s = lo;
    for (let i = lo; i < hi; i++) if (a[i] < a[hi]) { [a[i], a[s]] = [a[s], a[i]]; s++; }
    [a[s], a[hi]] = [a[hi], a[s]];
    if (s === k) return a[k];
    if (s < k) lo = s + 1; else hi = s - 1;    // *ఒక్క* వైపు కొనసాగడం
  }
  return a[k];
}
```

Quicksort రెండు వైపులా కొనసాగుతుంది. Quickselect **ఒక్క వైపు** – కాబట్టి $O(n \log n)$ కి బదులు **సగటున $O(n)$** : $n + n/2 + n/4 + \dots = 2n$.

9. Step – pivot ఎంపిక

ఆ "సగటున" అనే మాట ఒక ఊహ మీద ఆధారపడుతుంది: **pivot ప్రతిసారీ దాదాపు మధ్యలో పడుతుంది.**

సహజమైన ఎంపిక: `pivot = lo` (మొదటి మూలకం). సులభం, ఎప్పుడూ పనిచేస్తుంది.

కొలిచి చూద్దాం.

10. మూడో విరుపు – sorted input మీద 1,189 రెట్లు

Quickselect · k-వ అతి చిన్నది · ఎన్ని పోలికలు?

input	n	మొదటిది pivot	యాదృచ్ఛిక pivot	ఎన్ని రెట్లు
యాదృచ్ఛికం	1,000	3,862	2,120	1.8×
ఇప్పటికే sorted	1,000	3,75,249	5,018	74.8×
యాదృచ్ఛికం	4,000	14,055	15,330	0.9×
ఇప్పటికే sorted	4,000	60,00,999	20,160	297.7×

యాదృచ్ఛికం	16,000	48,866	72,909	0.7×
ఇప్పటికే sorted	16,000	9,60,03,999	80,723	1189.3×

SORTED INPUT · పోలికలు 3,75,249 → 60,00,999 → 9,60,03,999

n ని 4 రెట్లు పెంచితే పోలికలు **16 రెట్లు**. అదే చతుర్బుజ ప్రవర్తన, సందేహం లేదు.

కారణం: sorted array lo మొదటి మూలకం **ఎప్పుడూ అతి చిన్నది**. కాబట్టి ప్రతి విభజనలో ఒక్క మూలకం మాత్రమే వేరవుతుంది, మిగతా $n-1$ అలాగే ఉంటాయి. $n + (n-1) + (n-2) + \dots = \mathbf{n^2/2}$.

మరియు "ఇప్పటికే sorted" అనేది ఒక అరుదైన input కాదు – **అదే అత్యంత సాధారణమైనది**. Database నుంచి వచ్చిన data, timestamps, ids.

కానీ మొదటి, మూడో, అయిదో వరుసలు చూడండి

యాదృచ్ఛిక input మీద "మొదటిది pivot" వేగం – $0.7\times$ నుంచి $1.8\times$.

ఎందుకంటే యాదృచ్ఛిక data lo మొదటి మూలకం కూడా యాదృచ్ఛికమే, మరియు `rand()` పిలవడం ఒక అదనపు ఖర్చు.

అంటే యాదృచ్ఛిక pivot ఒక వేగ optimisation కాదు – అది ఒక భీమా. మీరు సగటు సందర్భంలో కొంచెం ఎక్కువ చెల్లించి, అత్యంత చెడ్డ సందర్భాన్ని **1,189 రెట్లు నుంచి** తొలగిస్తున్నారు.

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code

```
class Heap {
  #a = []; #c;
  constructor(cmp = (x, y) => x - y) { this.#c = cmp; } // cmp(x,y) < 0 → x పైన

  push(x) {
    const a = this.#a; a.push(x);
    let i = a.length - 1;
    while (i > 0) { // పైకి జరపడం
      const p = (i - 1) >> 1;
      if (this.#c(a[p], a[i]) <= 0) break;
      [a[p], a[i]] = [a[i], a[p]]; i = p;
    }
  }
  pop() {
    const a = this.#a;
    if (!a.length) return undefined;
    const top = a[0], last = a.pop();
    if (a.length) {
      a[0] = last;
      let i = 0;
      for (;;) { // కిందికి జరపడం
        const l = 2*i + 1, r = l + 1;
        let m = i;
        if (l < a.length && this.#c(a[l], a[m]) < 0) m = l;
        if (r < a.length && this.#c(a[r], a[m]) < 0) m = r;
        if (m === i) break;
        [a[m], a[i]] = [a[i], a[m]]; i = m;
      }
    }
    return top;
  }
}
// అతి పెద్ద k - **min**-heap, ఎందుకంటే మనం తొలగించేది అతి చిన్నది (§7)
function topKLargest(arr, k) {
  if (k <= 0) return [];
  const h = new Heap((x, y) => x - y); // min-heap
  for (const x of arr) {
    if (h.size < k) h.push(x);
    else if (x > h.peek()) { h.pop(); h.push(x); }
  }
  return h.toArray().sort((a, b) => b - a);
}
```

`pop()` లో **రెండు పిల్లలను** చూడాలని గమనించండి – కుడి బిడ్డని వదిలేస్తే heap నియమం విరుగుతుంది, మరియు §12 లో అది **150/150**.

12. 87,647 pushes · mutation testing · పనిని ఒక నియమంగా

ఎనిమిది నియమాలు: **heap** పరిమాణం సరిపోవాలి, **pop** క్రమం పెరుగుతున్న క్రమంలో, అవే మూలకాలు తిరిగి రావాలి, ఖాళీ **heap pop** → **undefined**, **topK** = **sort** చేసి మొదటి **k**, **topK** పొడవు **min(k, n)**, **quickselect** = **sort** చేసి **k**-వ మూలకం, **quickselect** పని **O(n)** గా ఉండాలి.

ఆ చివరి నియమం ముఖ్యమైనది – **Deep 01 lo** `steps ≤ ⌈log2(n+1)⌉` లాగా, ఇక్కడ **Big-O** ఒక **assertion**:

```
if (qs.cmps > 12 * n + 20)
    bad.push(`quickselect n=${n} కి ${qs.cmps} పోలికలు (పరిమితి ${12*n+20})`);
```

400 runs · 87,647 pushes · 16,000 top-K · 16,000 selects
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

heaps	16,000
pushes	87,647
pops	87,647
topks	16,000
selects	16,000
dupes	7,512
sorted	4,704
big	3,983

==== mutation testing ====

మార్పు లేని code	→ 0/400 విఫలం
topK lo min-heap కి బదులు max-heap	→ 150/150 విఫలం
ఉదా: topK([0,2,0,12,20,19,23], 5) = [23,12,2,0,0]	
topK lo `x > h.peek()` ని `x < h.peek()` గా	→ 150/150 విఫలం
push lo పైకి జరపడం తీసేస్తే	→ 150/150 విఫలం
ఉదా: pop క్రమం తప్పు: [0,10,5,12,14,19,19]	
pop lo కుడి బిడ్డని చూడకపోతే	→ 150/150 విఫలం
ఉదా: pop క్రమం తప్పు: [0,5,14,10,19,12,19]	
pop lo మూలాన్ని కిందికి జరపకపోతే	→ 150/150 విఫలం
quickselect lo యాదృచ్ఛిక pivot కి బదులు ఎప్పుడూ lo	→ 139/150 విఫలం
ఉదా: quickselect n=298 కి 40932 పోలికలు (పరిమితి 3596)	
quickselect lo `a[i] < a[hi]` ని `a[i] <= a[hi]` గా	→ 0/150 విఫలం

ఆ PIVOT వరుస - ఒక్క సరైనతన నియమమూ విఫలం కాలేదు

`pivot = lo` తో quickselect ఎప్పుడూ సరైన జవాబే ఇస్తుంది. కాబట్టి "జవాబు సరైనదా" అనే నియమాలన్నీ నెగ్గుతాయి.

దాన్ని పట్టుకున్నది పని-పరిమితి నియమం: $n = 298$ కి **40,932 పోలికలు**, పరిమితి 3,596.

సరైనతనం ఒక నియమం. వేగం ఇంకొక నియమం. రెండూ రాయకపోతే మీరు సగమే పరీక్షిస్తున్నారు.

మరియు ఆ నియమం మొదట **0/150** చూపించింది

నా arrays పొడవు 1-12. $n = 12$ కి చతుర్భుజం కూడా 144 పోలికలు - పరిమితి $12n+20 = 164$ కంటే తక్కువ.

100-400 పొడవు arrays (25% cases) జేడించాక - **139/150**.

Deep 01, 03, 05 - ఇప్పుడు నాలుగోసారి: ఒక mutation బతికితే, ముందు మీ data ని అనుమానించండి.

13. బతికిన మార్పు - నకళ్ళు మరియు మూడు-ముక్కల విభజన

`a[i] < a[hi]` ని `a[i] <= a[hi]` గా మార్చడం **0/150**. దాన్ని విడిగా పరీక్షించాను - నకళ్ళ యొక్క అత్యంత చెడ్డ సందర్భం మీద:

అన్నీ ఒకే విలువ ఉన్న array

n	`<` పోలికలు	`<=` పోలికలు	జవాబులు సరైనవా
100	3,774	3,725	రెండూ సరైనవి
400	60,099	59,900	రెండూ సరైనవి
1600	9,60,399	9,59,600	రెండూ సరైనవి

రెండూ చతుర్భుజం - మార్పు సమానమైనది, మరియు అసలు **CODE** కూడా ఇక్కడ విఫలం

$n = 1,600$ కి **9.6 లక్షల పోలికలు** - అంటే $n^2/2$. యాదృచ్ఛిక pivot ఉన్నా సరే.

ఎందుకంటే అన్ని విలువలూ సమానం అయితే, ఏ **pivot ఎంచుకున్నా** విభజన ఒక వైపు పడుతుంది. Pivot యాదృచ్ఛికత సహాయపడదు - సమస్య pivot కాదు, **విభజన**.

పరిష్కారం: **మూడు-ముక్కల విభజన** (Dutch national flag) - pivot కంటే చిన్నవి, **సమానమైనవి**, పెద్దవి అని మూడు భాగాలు. అప్పుడు సమానమైనవన్నీ ఒకేసారి తొలగిపోతాయి, మరియు అన్నీ-సమానం array **O(n)** అవుతుంది.

నేను దీన్ని ఈ doc lo అమలు చేయలేదు - కాబట్టి ఆ $O(n)$ కి నా దగ్గర సంఖ్య లేదు. కానీ పై పట్టిక **ఎందుకు అవసరమో** చూపిస్తుంది.

14. Follow-ups – heapify, median stream, k-way merge

"ఒక array ని heap గా మార్చాలంటే?"

n సార్లు `push` చేస్తే $O(n \log n)$. కానీ **heapify** $O(n)$: చివరి nonleaf node నుంచి మొదటికి వెళ్తూ ప్రతి దాన్నీ **కిందికి జరపడం**. ఎందుకు $O(n)$? ఎందుకంటే చాలా nodes ఆకుల దగ్గర ఉంటాయి, మరియు వాటిని జరపాల్సిన దూరం **సున్నా**. మొత్తం $n/2 \cdot 1 + n/4 \cdot 2 + n/8 \cdot 3 + \dots = 2n$. **నేను ఈ తేడాని కొలవలేదు.**

"ప్రవాహంలో median?"

రెండు heaps: ఎడమ సగానికి max-heap, కుడి సగానికి min-heap, పరిమాణాలు ఒక్క తేడాలో ఉంచడం.

Median = పెద్ద heap పైనున్నది (లేదా రెండింటి సగటు). ప్రతి చేర్పుకీ $O(\log n)$. §7 యొక్క అదే ఆలోచన రెండుసార్లు – **ప్రతి heap తను బయటికి పంపేదాన్ని పైన ఉంచుతుంది.**

"k sorted జాబితాలని కలపాలంటే?"

k-పరిమాణ min-heap, ప్రతి జాబితా నుంచి ప్రస్తుత మూలకం. మొత్తం N మూలకాలకి **$O(N \log k)$** – అన్నిటిని కలిపి sort చేస్తే $O(N \log N)$. $k \ll N$ అయితే ఇది గణనీయమైన తేడా.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **heapify** యొక్క $O(n)$ vs $O(n \log n)$ (పైన), **మూడు-ముక్కల విభజన** (§13), మరియు **heap తో Dijkstra** – Deep 05 lo నా Dijkstra $O(V^2)$, మరియు heap తో అది $O(E \log V)$ అవుతుంది; ఆ crossover ని నేను కొలవలేదు.

15. ఏమి నేర్చుకున్నాం

1. Heap sorted జాబితా కాదు – అది ఒక్క ప్రశ్నకే జవాబిస్తుంది. మరియు ఆ తక్కువ హామీయే $O(\log n)$ ఇస్తుంది.

2. Sorting మీకు అవసరం లేని దాన్ని లెక్కిస్తుంది. 10 లక్షల్లో అతి పెద్ద 100 కి: **289 ms vs 4.1 ms**, మరియు sort సమయం k తో మారనే లేదు.

3. కానీ k = 1 అయితే ఒక సాధారణ loop గెలుస్తుంది – 2.4 ms vs 5.8 ms. ఐదో doc lo నాలుగోసారి అదే నమూనా.

4. Heap పైన ఉండేది మీరు బయటికి పంపేది. అతి పెద్దవి కావాలంటే **min-heap** – max-heap వాడితే **65.8% తప్పు**, మరియు జవాబులో array యొక్క **అతి చిన్న మూలకం** ఉంటుంది.

5. యాదృచ్ఛిక pivot ఒక optimisation కాదు – ఒక బీమా. యాదృచ్ఛిక data మీద అది **కొంచెం నెమ్మది** ($0.7\times$), మరియు sorted data మీద **1,189 రెట్లు** వేగం.

6. "ఇప్పటికే sorted" అనేది ఒక అరుదైన input కాదు. Database queries, timestamps, ids – అన్నీ sorted గా వస్తాయి.

7. సరైనతనం ఒక నియమం; వేగం ఇంకొకటి. Pivot mutation ఒక్క సరైనతన నియమమూ విఫలం చేయలేదు. దాన్ని పట్టుకున్నది పని-పరిమితి.

8. మరియు ఆ నియమం కూడా data సరిపోకపోతే పనిచేయదు. $n \leq 12$ కి n^2 కూడా పరిమితి లోపలే – పెద్ద arrays జోడించాక $0/150 \rightarrow 139/150$.

9. కొన్నిసార్లు అసలు code కూడా విఫలం. అన్నీ-సమానం array మీద నా quickselect చతుర్భుజం – మరియు దానికి pivot కాదు, విభజన మార్పాలి.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Heap అనేది "నాకు మొత్తం క్రమం అవసరం లేదు" అని చెప్పే మార్గం.

మీరు ఎంత తక్కువ అడిగితే, అంత తక్కువ చెల్లిస్తారు:

మొత్తం క్రమం కావాలా $\rightarrow$ sort, $O(n \log n)$.

అతి పెద్ద k కావాలా $\rightarrow$ heap, $O(n \log k)$.

k-వ ఒక్క మూలకం కావాలా $\rightarrow$ quickselect, $O(n)$.

మూడూ ఒకే data మీద, మూడు వేర్వేరు ధరలు. మరియు తేడా algorithm lo కాదు – మీరు అడిగిన ప్రశ్నలో.